Models and Tools for Cyber-Physical Systems

Digital Written Exam, June the 16th 2025, 9:00-13:00

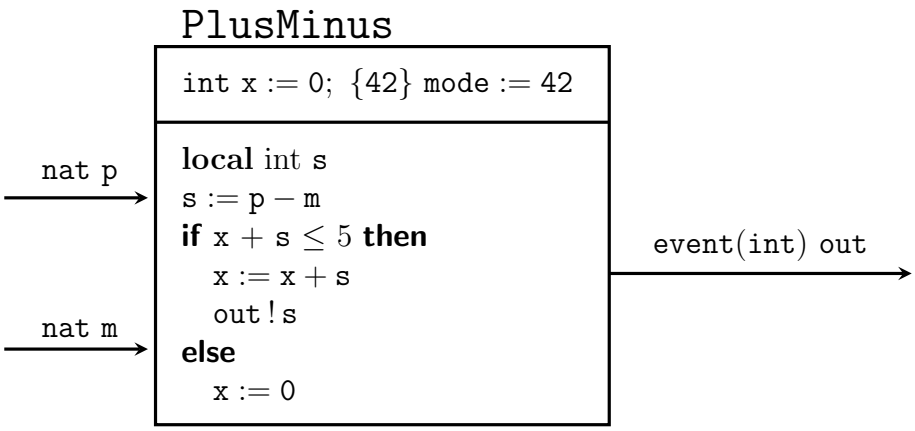
Please read the following before solving the exercises.

- This exam contains 6 exercises, worth 75 points in total. Each exercise is compulsory. The solution has to be composed in English. If you believe that the assignment wording is ambiguous or erroneous, then write down what additional assumption you are using and outline your reasons.
- In some exercises you are asked to extend incomplete UPPAAL and Python files. Solutions have to be uploaded to Digitaleksamen in the form of UPPAAL, Python, pdf or text files. For each of your solutions, ensure that it is clear which exercise is solved.
- The exam is open book, but asking/receiving help from others is not allowed.
- In case of emergencies: students can contact the instructors during the exam by approaching the study secretary.

Last but not least, good luck!

Exercise 1: Synchronous Components 10 Points

Consider the synchronous component `PlusMinus` given as a block diagram below.



Hint: Recall that `out ! s` is our notation for the event assignment `out := s`, and that $\perp$ denotes that an event is not present.

(a) Below we wrote the initial state and a sequence of inputs. Fill in the blanks (i.e., both the **missing states** between the arrows and the **missing outputs** on the arrows) to obtain a proper execution.

$$(0, 42) \xrightarrow{(3,0)/} \underline{\qquad\qquad} \xrightarrow{(3,0)/} \underline{\qquad\qquad}$$

$$\xrightarrow{(6,2)/} \underline{\qquad\qquad} \xrightarrow{(1,3)/} \underline{\qquad\qquad}$$

(b) Give a **sequence of <u>two consecutive</u> inputs** (i.e., corresponding to an execution of two steps) leading to the state $(-2, 42)$.
(You <u>do not</u> have to provide the intermediate states and outputs.)

(c) Describe the component `PlusMinus` as an extended state machine.
Use the mode of the state machine to capture the value of the state variable `mode`, while the state variable `x` should be updated using assignments in the mode switches.
Instead of the local variable `s`, you may write `p - m` everywhere.

..... Solution sketch .....

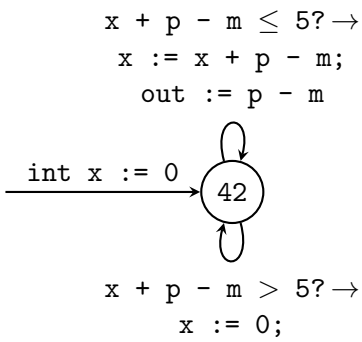
(a) $(0, 42) \xrightarrow{(3,0)/3} (3, 42) \xrightarrow{(3,0)/\perp} (0, 42) \xrightarrow{(6,2)/4} (4, 42) \xrightarrow{(1,3)/-2} (2, 42)$

(b) For instance: $(0, -1), (0, -1)$

Corresponding execution (just for the record; this was not required):

$$(0, 42) \xrightarrow{(0,-1)/-1} (-1, 42) \xrightarrow{(0,-1)/-1} (-2, 42)$$

(c) There is only one mode.



Exercise 2: Reachable States 10 Points

Consider again the synchronous component from Exercise 1.
Let φ_1 be the formula $x < 5$.

(a) How many reachable states does this component have?

(b) Is φ_1 an invariant? Shortly justify your answer.

(c) Is φ_1 an inductive invariant? Shortly justify your answer.

(d) Provide another formula φ_2 not equivalent to φ_1 that is an invariant.
(You do not have to justify why φ_2 is an invariant.)

(e) Provide another formula φ_3 not equivalent to φ_1 and φ_2 that is an inductive invariant.
Shortly justify why φ_3 is an inductive invariant.

.....Solution sketch .....

(a) There are infinitely many reachable states.
Specifically, the following states are reachable (note that this was not asked for):
$\{(v, 42) \in \mathbb{Z}^2 \mid v \leq 5\}$ (Note that v can be negative.)

(b) No, because the state $(5, 42)$ is reachable but does not satisfy φ_1.

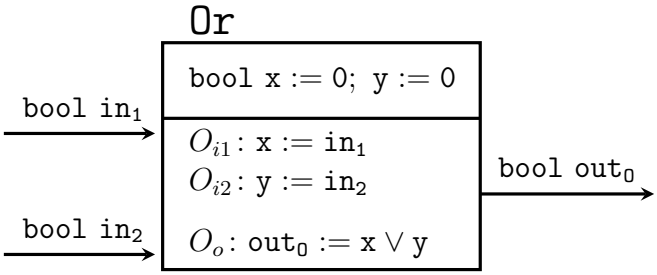
(c) No, because φ_1 is already not an invariant.

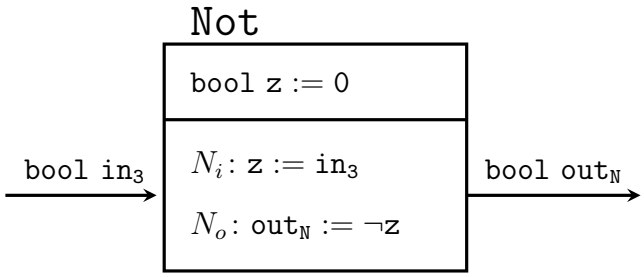
(d) An example is $\mathtt{x} \leq 6$.

(e) An example is $\mathtt{x} \leq 5 \wedge \mathtt{mode} = 42$.
This formula describes the set of reachable states, and the set of reachable states is an inductive invariant.

Exercise 3: Asynchronous Components 10 Points

Consider the asynchronous components `Or` and `Not` given as block diagrams below. Here, $\vee$ denotes the Boolean disjunction ("or") and $\neg$ denotes the Boolean negation ("not").





Consider the parallel composition given by the following description:

$$\texttt{Or}[\texttt{out}_0 \mapsto \texttt{temp}] \quad | \quad \texttt{Not}[\texttt{in}_3 \mapsto \texttt{temp}] \quad \backslash \quad \texttt{temp}$$

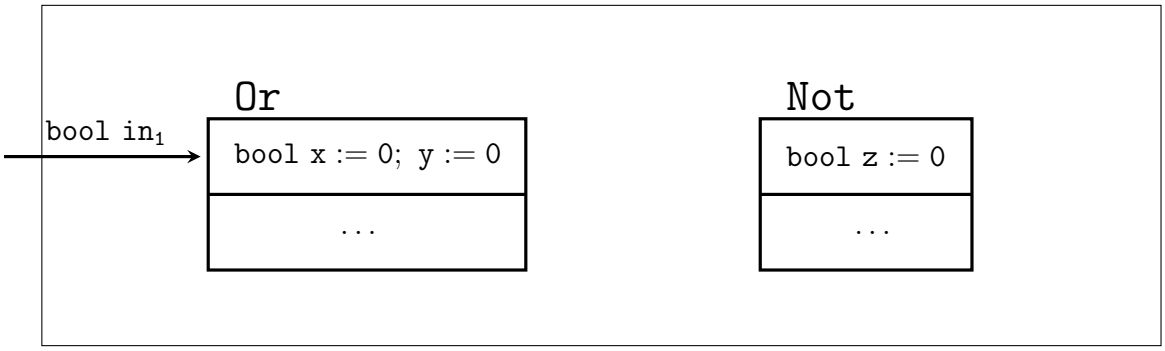
(a) Below is a partial block diagram for the parallel composition. Connect the inputs and outputs of the components correctly with edges/arrows. Inputs and outputs of the composition should leave the outer block; we have given an example for the input $\texttt{in}_1$.

(b) Annotate the edges you drew with the channel types and names.

(c) Give all input tasks, internal tasks, and output tasks of the parallel composition.

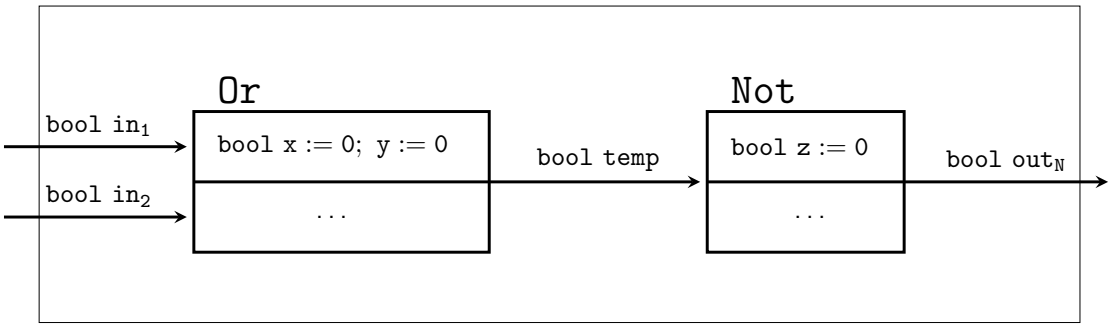
(d) Give an execution of the parallel composition for the inputs $\texttt{in}_1 = 1$ and $\texttt{in}_2 = 0$.

Hint: The local variable `temp` is not part of the state. Also, remember that executions for asynchronous components perform exactly one task in each step. Input tasks have the form $s \xrightarrow{\texttt{i}?v} s'$, internal tasks have the form $s \xrightarrow{\varepsilon} s'$, and output tasks have the form $s \xrightarrow{\texttt{o}!v} s'$, where v is a value, and `i` and `o` are input and output variables, respectively.



Solution sketch

(a) and (b)



(c)

$$\begin{aligned}
\text{input: } & \mathtt{x} := \mathtt{in}_1 \quad (= O_{i1}) \\
& \mathtt{y} := \mathtt{in}_2 \quad (= O_{i2}) \\
\text{internal: } & \mathtt{local\ temp} := \mathtt{x} \vee \mathtt{y};\ \mathtt{z} := \mathtt{temp} \quad (\text{or shortened: } \mathtt{z} := \mathtt{x} \vee \mathtt{y}) \\
\text{output: } & \mathtt{out}_\mathtt{N} := \neg \mathtt{z} \quad (= N_o)
\end{aligned}$$

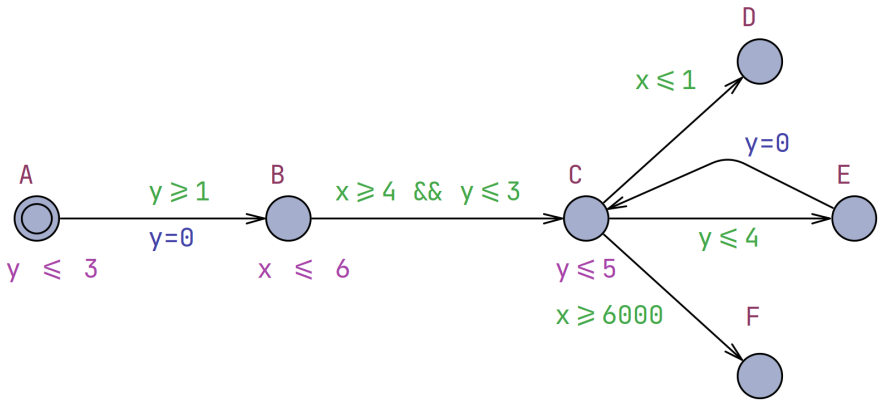
(d) We use the state order $(\mathtt{x}, \mathtt{y}, \mathtt{z})$.

We only have to apply the two input tasks (it was not required to produce an output):

$$(0,0,0) \xrightarrow{\mathtt{in}_1\,?\,1} (1,0,0) \xrightarrow{\mathtt{in}_2\,?\,0} (1,0,0)$$

Exercise 4: Exploration of Timed Automata 10 Points

Consider the timed automaton $\mathcal{A}$ below with two clocks x and y.



(a) Which of the three locations D, E and F are reachable from the initial state (A, $\mathrm{x} = 0, \mathrm{y} = 0$)?

(b) For each of these three goal locations that is reachable, provide a timed transition sequence that leads to the location from the initial state.

(c) For each of the three goal locations that is reachable, what is the fastest time of reaching that location? Provide a timed transition sequence as a witness.

(d) Describe, using difference constraints or its graphical representation, the reachable zones upon entry and after delay for the locations A, B and C.

(e) For each of the three goal locations that is NOT reachable, investigate whether it is possible to weaken the guard leading to the location, so that the location becomes reachable.
NOTE: y<=7 is weaker than y<=5. Similarly, y>=2 is weaker than y>=4.

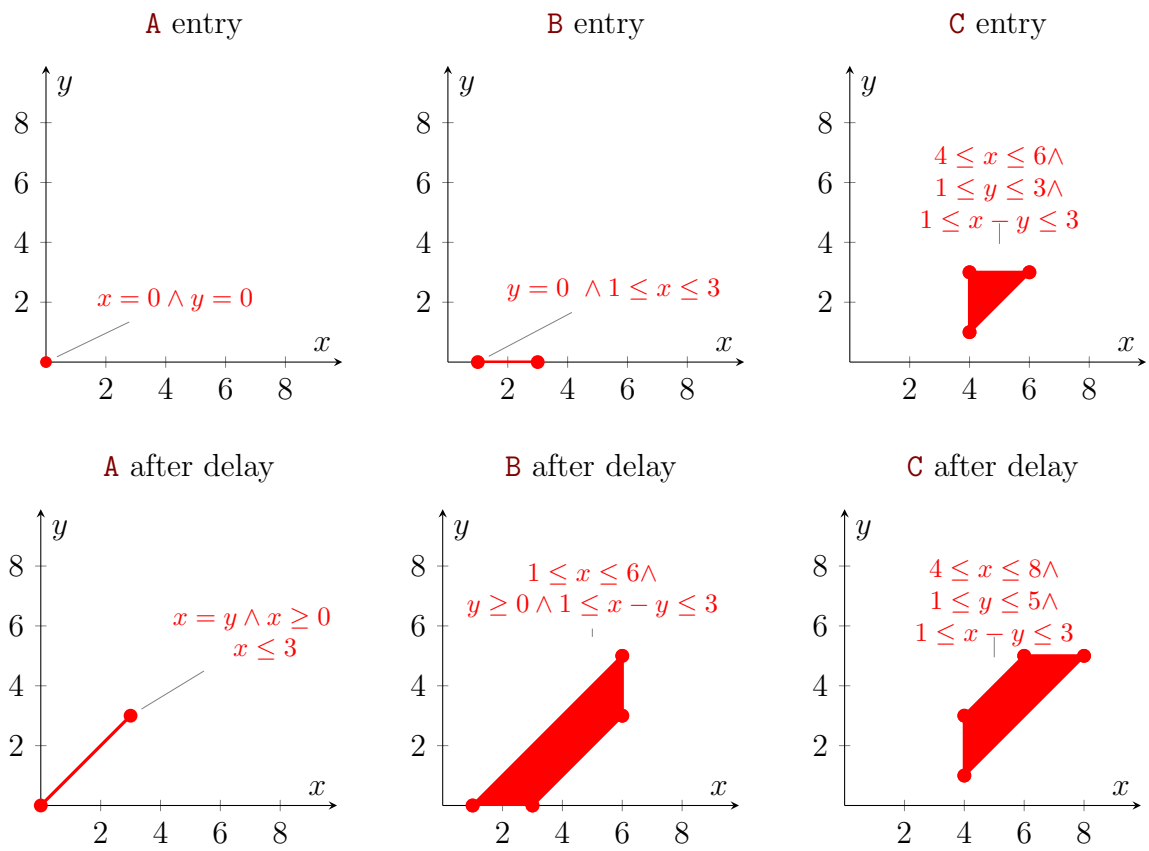
Solution sketch

(a) Both E and F are reachable.

(b) let $(\mathtt{A}, x = 0, y = 0) \xrightarrow{1} (\mathtt{A}, x = 1, y = 1) \rightarrow (\mathtt{B}, x = 1, y = 0) \xrightarrow{3} (\mathtt{B}, x = 4, y = 3) \rightarrow (\mathtt{C}, x = 4, y = 3) \rightarrow (\mathtt{E}, x = 4, y = 3)$
we can extend the previous execution leading to E as follows
$(\mathtt{A}, x = 0, y = 0) \xrightarrow{1} (\mathtt{A}, x = 1, y = 1) \rightarrow (\mathtt{B}, x = 1, y = 0) \xrightarrow{3} (\mathtt{B}, x = 4, y = 3) \rightarrow (\mathtt{C}, x = 4, y = 3) \rightarrow (\mathtt{E}, x = 4, y = 3) \xrightarrow{6000-4} (\mathtt{E}, x = 6000, y = 5999) \rightarrow (\mathtt{C}, x = 6000, y = 0) \rightarrow (\mathtt{F}, x = 6000, y = 0)$

(c) The fastest time to reach E is 4 time units, and the fastest time to reach F is 6000 time units. The witnessing sequences are above.

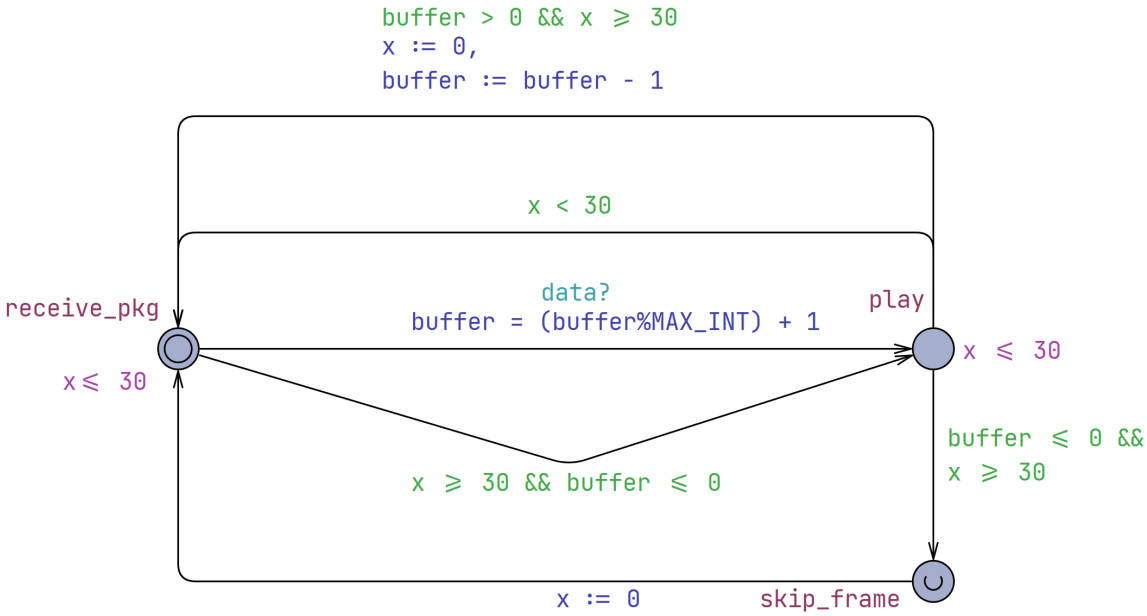
(d) The reachable zones upon entry and after delay:



(e) Changing the guard to $x \leq 4$ will make D reachable.

Exercise 5: Multimedia Streaming Protocol 15 Points

We consider a simplified multimedia streaming protocol modeled in UPPAAL. In our streaming protocol a component `device` sends `data` to a multimedia `player` component. The timed automaton for the multimedia `player` is given below.



When the component `player` receives data over the channel `data`, it increases the integer variable `buffer`, indicating that there are frames which can be played. The component `player` can receive several data packages, which will be stored in the buffer. The component `player` will play a frame every 30 milliseconds. If no frame data is available, i.e., `buffer <= 0`, the component `player` will skip the frame. Note that this will cause loss in media quality. The full UPPAAL model `streaming-protocol.xml` can be downloaded from Digital Exam. This initial model defines the concurrent system `device|player`.

(a) We want to know whether the system `device|player` is of high quality or skips frames. Provide a UPPAAL query to check that the system does not skip frames.

(b) The system includes a clock `time`. According to the clock `time`, when is the earliest point in time where a frame is skipped? Provide an execution as a witness. You may submit a screenshot of the symbolic simulator.

(c) Modify the component `device` to create the component `device2` such that the system `device2|player` skips no frames. Provide a screenshot of the component `device2`. Upload your model under the name `streaming-protocol-2.xml` to Digital Exam. The model should include the query from above that can be verified.

(d) For the the system `device2|player` provide a UPPAAL query to find the biggest reachable value for the variable `buffer` when the clock time is less than a second, i.e., `time <= SECOND`. What is the value? Note: If computation in UPPAAL is taking too long, ensure that the option *diagnostic trace* is set to *none*. Hint: have a look

at UPPAAL's query language. `https://docs.uppaal.org/language-reference/query-syntax/symbolic_queries/`.

(e) The current system has a `MAX_MEMORY` of 16GB. We would like to produce a more affordable model with a usage of at most 4GB in a `SECOND`.

(i) If the system `device2|player` already satisfies this property, you can re-use the component `device2` here.

If the system `device2|player` does not satisfy this requirement, modify the component `device2` to create the component `device3` such that the system `device3|player` skips no frames and the buffer size is below 4GB when the clock satisfies `time <= SECOND`. Provide a screenshot of the component `device3`.

(ii) Provide a UPPAAL query to verify that the buffer is always smaller than 4GB (4000) when the clock `time` is less than 1 `SECOND`.

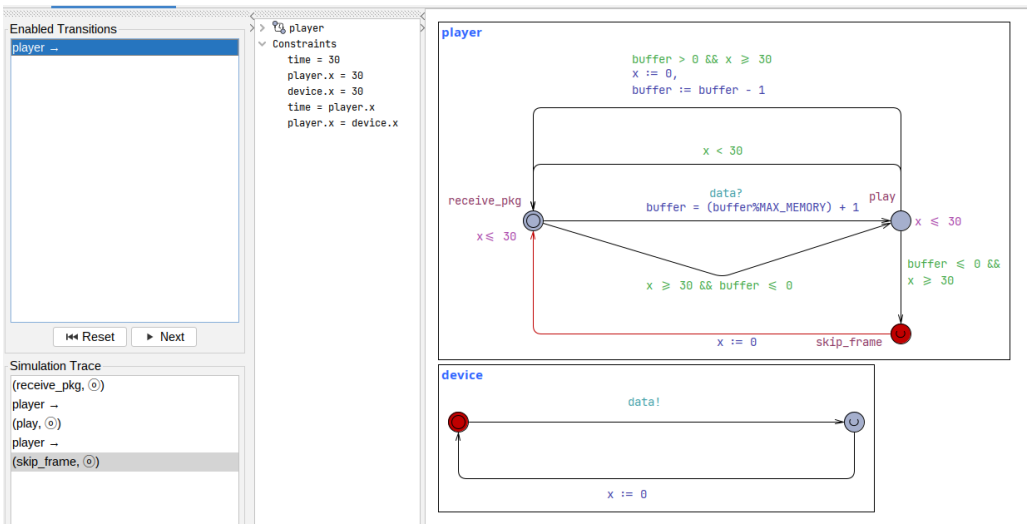
Upload your model to Digital Exam under the name `streaming-protocol-3.xml`. The model should include the required queries.

Solution sketch

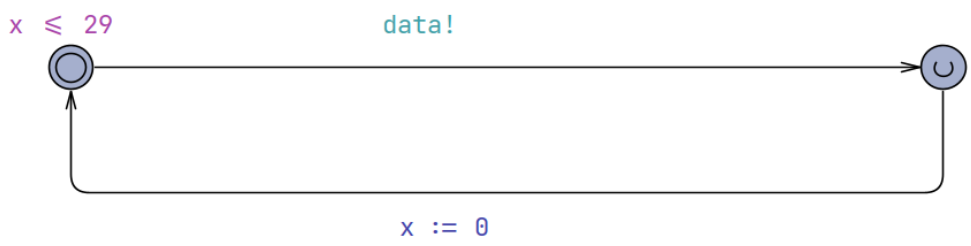
UPPAAL solution file `streaming-protocol-sol.xml`

(a) `A[] not player.skip_frame`

(b) Use options → diagnostic trace → fastest. See the counter-example for the query `A[] not player.skip_frame`. A screenshot:

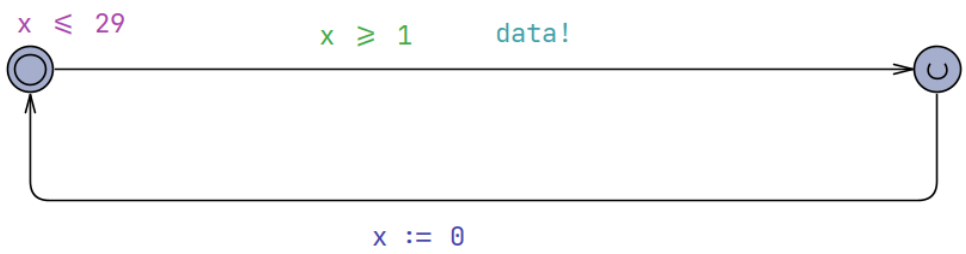


(c) Component `device` can delay forever in the initial location without sending any data. This can be fixed by adding an invariant, e.g.:



(d) `sup{time <= SECOND}: player.buffer`

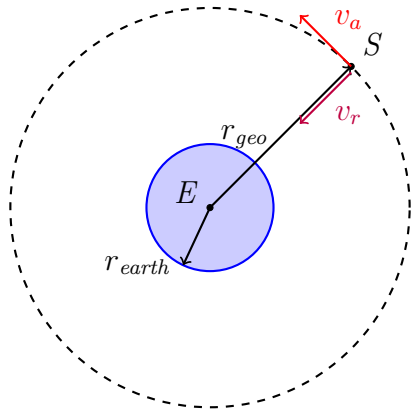
(e) (i) In the solution `device2`, sending data was immediate (no time). This would allow the variable `buffer` to increase. Adding a guard to emulate the cost of sending data will decrease the size of the buffer, e.g.:



(ii) `A[] player.buffer <= 4000 || time >= SECOND`

Exercise 6: Satellite 20 Points

We study a satellite (S) that is supposed to be on a geostationary orbit, i.e., at distance r_{geo} from earth's center (E). As long as the satellite stays at this distance, it remains at the same point in the sky for a person on earth's surface. That is, the earth and the satellite rotate with the same angular velocity v_a, see picture below.



We model the satellite by its radial distance r to earth's center and its radial velocity v_r. The satellite is subject to friction and is equipped with boosters to ensure that it can stay on the geostationary orbit. The differential equations are given by

$$\frac{d}{dt}r(t) = v_r(t) \qquad \frac{d}{dt}v_r(t) = -\frac{GM}{r^2} + \frac{L^2}{m^2r^3} - kv_a + F, \tag{1}$$

where F is the force due to the boosters, while G is the gravitational constant, M the mass of the earth, m the mass of the satellite, k the friction coefficient, and L the angular momentum which is assumed to be constant to simplify things. At last, the angular velocity v_a at radial distance r can be described by the formula

$$v_a = \frac{L}{mr}$$

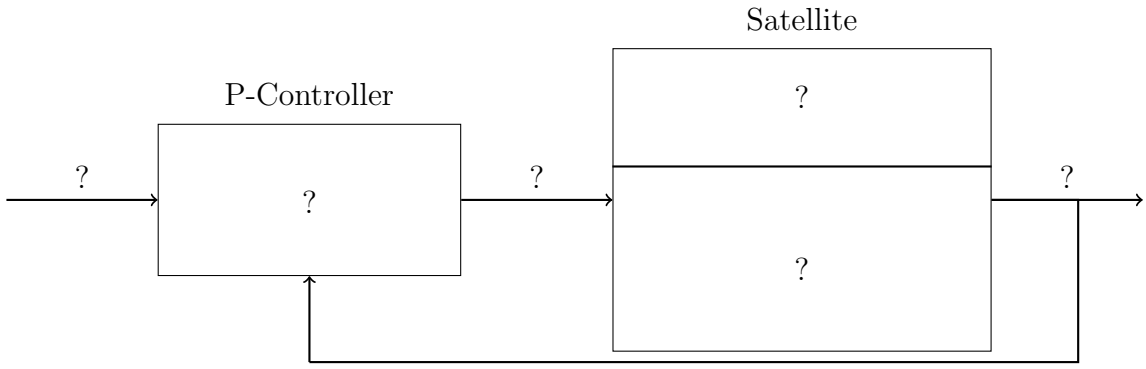
All above constants are specified in the provided Python script `Exam25.py`. Here, we mention the radius of the earth and the radius of the geostationary orbit in meter:

$$r_{earth} = 6.378 \cdot 10^6 \qquad r_{geo} = 42.164 \cdot 10^6$$

(a) Argue that the system is Lipschitz continuous on domain $r \geq r_{earth}$ if no control is being applied, i.e., $F = 0$. To this end:

- Use that sums of Lipschitz continuous functions are Lipschitz continuous.
- Derive a Lipschitz constant for r^{-n} on domain $r \geq r_{earth}$. Exploit here that the Lipschitz constant is the maximal absolute derivative value of a differentiable function across its domain. [Hint: recall that the derivative of x^{-n} is $-nx^{-(n+1)}$, yielding the absolute derivative value of $n|x|^{-(n+1)}$ at point x.]

(b) Measuring r, we wish to compute F using a P-controller that keeps the satellite at the geostationary orbit r_{geo}. Complete the block diagram below realizing this.



For the following three questions, you will need to complete the provided Python script `Exam25.py`. You can find the Python script in Digital Exam. After finishing this exercise, please upload your modified Python script in a file with name `Satellite.py`, also in case you could not solve all questions.

(c) Implement the `f` method in the provided Python script so that the overall program simulates an uncontrolled satellite which starts at time $t = 0$ on the geostationary orbit with zero radial velocity. Estimate the time of impact by plotting the simulation. How could one calculate satellite's speed at impact? [Hint: Pay attention, there is v_r and v_a.][1]

(d) Extend the implementation from the foregoing exercise to a P-controller (that is, you should get the result of the foregoing exercise when $K_P = 0$). Afterwards, tune the proportional gain K_P so that the satellite keeps within 5 meters of r_{geo}.[2] Describe the quality of your controller.

(e) Extend the P-controller from the foregoing exercise to a PID-controller. Extend the method `f` in the Python script accordingly. Tune the gains so that the satellite attains the geostationary orbit within 10 seconds, while remaining within a distance of 5 meters. Assess the quality of the PID controller.

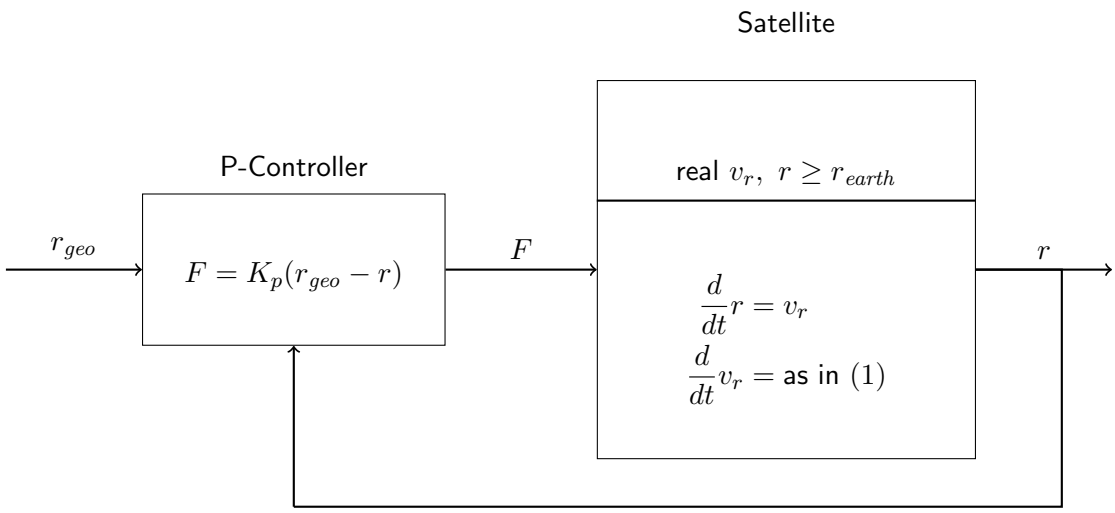
.....Solution sketch .....

(a) The absolute value of the derivative of r^{-n}, given by $|-nr^{-(n+1)}| = nr^{-(n+1)}$, shows that $n\,r_{earth}^{-(n+1)}$ is a Lipschitz constant of r^{-n} on the domain (note that $r^{-(n+1)}$ is monotonic decreasing). Since $\frac{d}{dt}v_r$ is a weighted sum of r^{-1}, r^{-2} and r^{-3}, this shows the claim.

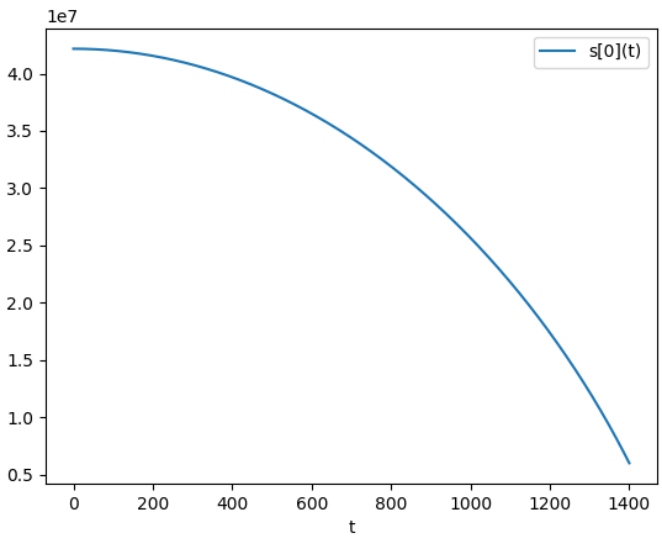
(b) The block diagram is completed as follows.

[1] For those who are curious, our model is strongly simplified because the actual friction coefficient will increase when the satellite approaches the earth. Likewise, the angular moment L too depends on the friction, hence is not really a constant. However, for $r \approx r_{geo}$ the model is realistic.

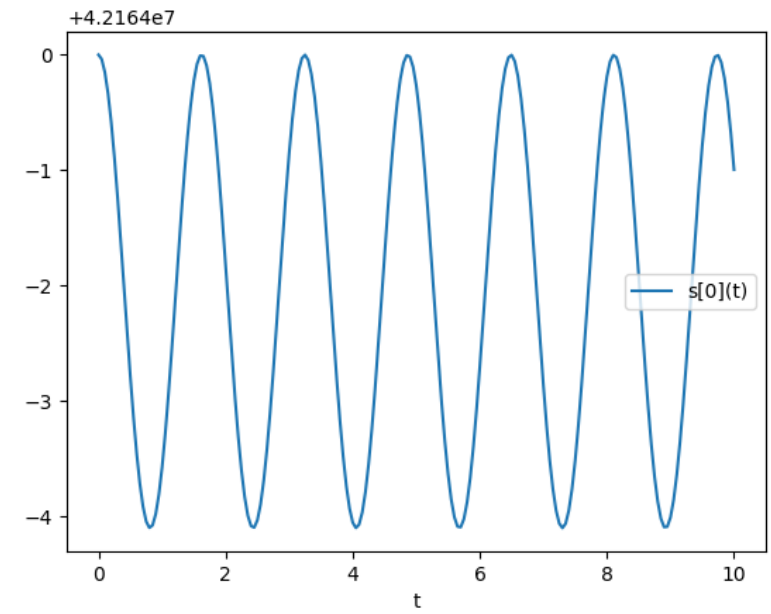
[2] The constants in `Exam25.py` are chosen such that the unit of r is meter.



(c) For formulas and code, please see e). The figure below plots r in case of no control, i.e., $K_P = K_D = K_I = 0$. It can be noted that the satellite crashes after around $\tau = 1400$ seconds. Pay attention that the crash occurs when $r = r_{earth}$ and not when $r = 0$. The impact speed calculates as $\sqrt{v_a^2(\tau) + v_r^2(\tau)}$, where $v_a(\tau) = L/mr(\tau)$.

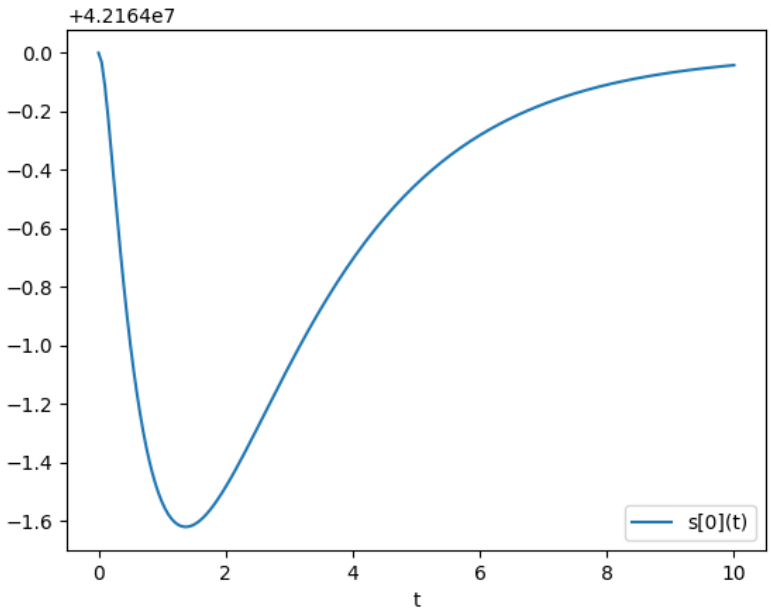


(d) The figure below plots r in case of a P-controller with $K_P = 15$. This is a considerable better outcome than the crash scenario where no control is present. The choice of K_P also makes sure that the satellite remains within a five meter distance to the geostationary orbit. It can be however noted that the control is not stabilizing the satellite at the geostationary orbit.



(e) The derivative error is equivalent to $\frac{d}{dt}e(t) = \frac{d}{dt}(r_{geo} - r(t)) = -v_r(t)$. With this, we can implement the derivative part of the PD-controller. Instead, the integral error $I(t) = \int e(t)\mathrm{d}t$ enjoys the differential equation $\frac{d}{dt}I(t) = e(t) = r_{geo} - r(t)$. This results in an additional state variable (so we need to adapt the initial state declaration in `main` as well), which we can use to define the integral part of the PID-controller.

The figure below plots r and is obtained for $K_P = 15$, $K_D = 10$ and $K_I = 5$. Overall, it can be noted that the PID controller provides a good performance, stabilizing the satellite at the geostationary orbit within 10 seconds.



```python
def f(s, t):
    # Constants, do not change
    k = 1.00e-2 # friction coefficient
    G = 6.67430e-11  # gravitational constant (m^3 kg^-1 s^-2)
    M = 5.972e+24  # mass of the Earth (kg)
    m = 1000  # mass of the satellite (kg)
    ref = 42.164e+6 (m)  # geostationary orbit radius (m)
    L = 1.2963840040e+14 # angular momentum (kg m^2 s^-1)

    e = (ref-s[0])
    Kp = 15
    Kd = 10
    Ki = 5
    #F = Kp*e # P
    #F = Kp*e - Kd*(s[1]) # PD

    F = Kp*e - Kd*(s[1]) + Ki*s[2] # PID

    r=s[0]
    v_r=s[1]

    v_angular = L/(m*r); # angular velocity of satellite (m/s)
    der_r = v_r
    der_v_r = -G*M/(r**2) + L**2/(m**2 * r**3) - k*v_angular + F
    der_I = ref - r

    return [der_r, der_v_r, der_I]

def main():
    t0 = 0 # start time
    R_geo = 42.164e+6 # geostationary orbit
    # r, v_r, I (integral error)
    x0 = [R_geo, 0, 0]
    T = 1400 # crash at around 1400 s
    #T = 10 # time horizon of 10 s
    ...
```